Fruit Into Baskets

Two approaches, visualized execution, and the core logic to remember

Brute Force · O(n²) | Sliding Window · O(n)

Reference notes · FruitIntoBasckets.java

Problem Statement

A row of fruit trees: `fruits[i]` is the fruit type of tree `i`. You have **two baskets**, each holding a **single type**. Starting from any tree, pick one fruit from every tree moving right, and stop at the first tree whose fruit fits in neither basket. Return the maximum number of fruits you can pick.

fruits = [1,2,3,2,2] → **longest run with at most 2 distinct types** → **Output: 4 ([2,3,2,2])**

In plain words: find the longest contiguous subarray containing **at most 2 distinct values**. "Two baskets" is just a story for `k = 2`.

Approach Comparison

Approach	Time	Space	Core Idea
Brute Force	O(n²)	O(1)	Restart a fresh map at every start, stop at the 3rd fruit type
Sliding Window	O(n)	O(1)	Expand right, shrink left when > 2 types, record every step

Big picture: the map is both a fruit counter and a type counter — `map.size()` is the number of distinct fruit types in the window. The sliding window maintains it incrementally instead of rebuilding it for every start tree. Space is O(1) because the map never holds more than 3 keys.

Heads-up: the code uses `java.util.Map` and `java.util.HashMap` — make sure both imports are present in the source file.

1 · Brute Force

O(n²) time **O(1) space**

For each start tree i → **Walk right, count types in map** → **size > 2? break** → **else record j - i + 1**

```java
// try every start, keep going until a 3rd fruit type shows up
private static int usingBruteForce(int[] nums) {
    int maxLength = 0;
    int k = 2;

    for (int i = 0; i < nums.length; i++) {
        Map<Integer, Integer> map = new HashMap<>();

        for (int j = i; j < nums.length; j++) {
            int fruit = nums[j];
            map.put(fruit, map.getOrDefault(fruit, 0) + 1);

            if (map.size() > k) {
                break;
            }

            int currentLength = j - i + 1;
            maxLength = Math.max(maxLength, currentLength);
        }
    }
    return maxLength;
}
```

Recall trick: "Try every start, keep going until a 3rd fruit type shows up." Correct, but rebuilds the map from scratch for every start — O(n²) in the worst case.

2 · Sliding Window (Variable Size)

O(n) time **O(1) space**

Expand right: add fruit to map → **size > 2? shrink from low** → **count hits 0? remove key** → **record high - low**

```java
// expand right; more than 2 types? shrink left; record the length every step
private static int usingSlidingWindow(int[] nums) {
    int high = 0;
    int low = 0;
    int maxLength = Integer.MIN_VALUE;
    int k = 2;
    Map<Integer, Integer> map = new HashMap<>();

    while (high < nums.length) {
        int highElement = nums[high];

        map.put(highElement, map.getOrDefault(highElement, 0) + 1);
        high++;

        while (map.size() > k) {
            int lowElement = nums[low];
            map.put(lowElement, map.getOrDefault(lowElement, 0) - 1);

            if (map.get(lowElement) == 0) {
                map.remove(lowElement);
            }
            low++;
        }

        int currentLength = high - low;
        maxLength = Math.max(maxLength, currentLength);
    }
    return maxLength;
}
```

Trace for `fruits = [1,2,3,2,2]`:

high (after++)	fruit added	map after shrink	low	window length	maxLength
1	1	{1:1}	0	1	1
2	2	{1:1, 2:1}	0	2	2
3	3	{2:1, 3:1}	0→1	2 (3 types → dropped fruit 1)	2
4	2	{2:2, 3:1}	1	3	3
5	**2**	**{2:3, 3:1}**	**1**	**4**	**4**

At most vs exactly: this problem is *at most* 2 distinct, so the length is recorded **unconditionally** after the shrink loop. In *Longest Substring with K Distinct Characters* it had to be *exactly* k, so the length was recorded only when `map.size() == k`. Same skeleton, one different condition.

TL;DR — For Future Me

- **Best solution:** variable-size sliding window + frequency map, O(n) — "longest subarray with at most 2 distinct values."
- **Brute force** is O(n²) — rebuilds the map for every start tree.
- **Must remember:** remove a key when its count reaches 0 — `map.size()` *is* the distinct-type count.
- **Pattern:** this is *Longest Substring with At Most K Distinct Characters* with `k = 2`, dressed up as a story. Spot the "two baskets = two distinct types" translation and it's the same template.